9i. Static Vectors for Small Collections

Martin Alfaro
PhD in Economics

INTRODUCTION

Frequent vector creation can quickly become a performance bottleneck due to its memory-allocation overhead. The issue has far-reaching implications, since vector creation doesn't just occur when a vector is explicitly defined. It also happens internally in various scenarios, such as when referencing a slice like `x[1:2]` in `sum(x[1:2])` or when computing intermediate results on the fly like `x .* y` in `sum(x .* y)`. For instance, referencing a slice like `x[1:2]` within an expression like `sum(x[1:2])`, or computing intermediate results on the fly like `x .* y` in `sum(x .* y)`, often triggers hidden allocations behind the scenes.

The current section introduces a strategy to address this issue, while retaining the convenience of vectors for expressing collections. It leverages the so-called **static vectors**, provided by the `StaticArrays` package. Unlike built-in vectors, which are allocated on the heap, static vectors are stack-allocated.

Internally, static vectors are built on top of tuples. Given the small capacity of the stack, this determines that **static vectors are only suitable for collections comprising a few elements**. As a rule of thumb, **you should use static vectors for collections with up to 50 elements**. Exceeding this threshold can lead to increased overhead, potentially offsetting any performance benefits or even resulting in a fatal error. [1]

Static vectors offer additional benefits relative to tuples. Firstly, they maintain their performance benefits, even at sizes where tuples typically degrade. Secondly, their manipulation is closer to regular vectors, making them more convenient to work with.

The `StaticArrays` package also supports other array types (including matrices) and have mutable variants. The latter make static vectors more flexible than tuples, which are only available in an immutable form. It's worth indicating though that, while the mutable version provides performance benefits relative to regular vectors, the immutable option still offers the best performance.

> ⚠️ **Warning!**
>
> **The entire section assumes all collections are small**, so all the benefits highlighted are contingent upon this assumption.
>
> We also suppose that the `StaticArrays` package has been loaded by executing `using StaticArrays`, thus omitting this command from each code snippet.

PERFORMANCE GAINS OF TUPLES FOR SMALL COLLECTIONS

Since static vectors are built on top of tuples, we begin by illustrating the advantages of tuples over vectors. In particular, we point out that these advantages only materialize when the collections are small. This is because unlike heap-allocated objects like vectors, tuples live on the stack, which has very limited space.

Recall that tuples are immutable and have a fixed size. This allows the compiler to aggressively optimize operations on them. However, as the number of elements grows, tuple-based computations become prohibitively expensive. Beyond a certain size, performance degrades sharply. In fact, once the tuple exceeds a particular threshold, the computation may fail to complete at all, causing the program to stall.

SMALL COLLECTION

```julia
_size  = 10
vector = rand(_size)
tuple  = Tuple(vector)

foo(x) = sum(x .* x)
```

```
julia> foo(vector)
  34.806 ns (2 allocations: 144 bytes)
julia> foo(tuple)
  1.742 ns (0 allocations: 0 bytes)
```

MEDIUM COLLECTION

```julia
_size  = 200
vector = rand(_size)
tuple  = Tuple(vector)

foo(x) = sum(x .* x)
```

```
julia> foo(vector)
  109.528 ns (2 allocations: 1.625 KiB)
julia> foo(tuple)
  120.992 ns (0 allocations: 0 bytes)
```

BIG COLLECTION

```
_size  = 2_000
vector = rand(_size)
tuple  = Tuple(vector)

foo(x) = sum(x .* x)
```

```
julia> foo(vector)
  1.217 μs (3 allocations: 15.688 KiB)
julia> foo(tuple)
  2.306 μs (0 allocations: 0 bytes)
```

BIG COLLECTION - PROGRAM STALLED

```
_size  = 10_000
vector = rand(_size)
tuple  = Tuple(vector)

foo(x) = sum(x.* x)
```

```
julia> foo(vector)
  8.197 μs (3 allocations: 78.188 KiB)
julia> foo(tuple)
# program keeps working indefinitely
```

Large collections can also trigger a different kind of failure: a Stack Overflow. This typically occurs when the program exhausts the available stack space. In this scenario, instead of the computation hanging indefinitely, Julia aborts execution and reports an error.

STACK OVERFLOW

```
function foo(n)
    if n == 0
        ()
    else
        (n, foo(n-1)...)
    end
end
```

```
julia> foo(50_000)
ERROR: StackOverflowError
```

Next, we turn to the package `StaticArrays`. This provides the same advantages as tuples, also extending their functionalities.

CREATING STATIC VECTORS

The package `StaticArrays` includes several variants of static arrays. Our primary focus is on the type `SVector`, whose objects will be referred to as SVectors for simplicity. Similar to tuples, **SVectors are immutable**, meaning that their elements can't be added, removed, or modified.

There are two approaches to creating an SVector, each serving a distinct purpose: the first one creates an SVector through literal values, while the other converts a regular vector into an SVector. The implementation of each approach is illustrated below.

VIA LITERAL VALUES

```
# all 'sx' define the same static vector '[3,4,5]'


sx = SVector(3,4,5)
sx = SVector{3, Int64}(3,4,5)
sx = SA[3,4,5]
sx = @SVector [i for i in 3:5]
```

```
julia> sx
3-element SVector{3, Int64} with indices SOneTo(3):
 3
 4
 5
```

VIA EXISTING VECTOR

```
# all 'sx' define a static vector with same elements as 'x'
x = collect(1:10)

sx = SVector(x...)
sx = SVector{length(x), eltype(x)}(x)
sx = SA[x...]
sx = @SVector [a for a in x]
```

```
julia> sx
10-element SVector{10, Int64} with indices SOneTo(10):
  1
  2
  3
  ⋮
  9
 10
```

Of these approaches, we'll primarily rely on the function `SVector`, occasionally employing `SA` for indexing purposes.[2] Note that the use of splat operator `...` is necessary when a regular vector must be turned into an SVector. This operator transforms a collection into a sequence of arguments. Thus, `foo(x...)` is equivalent to `foo(x[1], x[2], x[3])` given a vector or tuple `x` with 3 elements.

Regarding slices of SVectors, the **method used for indexing determines whether the resulting slice is a regular vector or an SVector**: a slice remains an SVector when indices are provided as SVectors, whereas the slice becomes a regular vector when indices are ranges or regular vectors. The

sole exception to this rule is when the slice references the whole object (i.e., `sx[:]`), in which case an SVector is returned.

SLICES AS SVECTORS

```
x      = collect(3:10)
sx     = SVector(x...)

# both define static vectors
slice1 = sx[:]
slice2 = sx[SA[1,2]]        # or slice2 = sx[SVector(1,2)]
```

```
julia> slice1
8-element SVector{8, Int64} with indices SOneTo(8):
  3
  4
  ⋮
  9
 10
julia> slice2
2-element SVector{2, Int64} with indices SOneTo(2):
 3
 4
```

WRONG APPROACH

```
x      = collect(3:10)
sx     = SVector(x...)

# both define and ordinary vector
slice1 = sx[1:2]
slice2 = sx[[1,2]]
```

```
julia> slice
2-element Vector{Int64}:
 3
 4
```

SVECTORS DON'T ALLOCATE MEMORY AND ARE FASTER

One key advantage of SVectors is that they're internally implemented on top of tuples. Consequently, SVectors don't allocate memory.

COPY

```julia
x   = rand(10)


function foo(x)
    a = x[1:2]              #   allocation (copy of slice)
    b = [3,4]               #   allocation (vector creation)

    sum(a) * sum(b)         # 0 allocation (scalars don't allocate)
end
```

```
julia> @btime foo($x)
  42.844 ns (3 allocations: 128 bytes)
```

VIEW

```julia
x   = rand(10)


@views function foo(x)
    a = x[1:2]              # 0 allocation (view of slice)
    b = [3,4]               #   allocation (vector creation)

    sum(a) * sum(b)         # 0 allocation (scalars don't allocate)
end
```

```
julia> @btime foo($x)
  17.411 ns (1 allocations: 48 bytes)
```

TUPLE

```julia
x   = rand(10)
tup = Tuple(x)

function foo(x)
    a = x[1:2]              # 0 allocation (slice of tuple)
    b = (3,4)               # 0 allocation (tuple creation)

    sum(a) * sum(b)         # 0 allocation (scalars don't allocate)
end
```

```
julia> @btime foo($tup)
  1.909 ns (0 allocations: 0 bytes)
```

STATIC VECTOR

```
x   = rand(10)
sx  = SA[x...]

function foo(x)
    a = x[SA[1,2]]          # 0 allocation (slice of static array)
    b = SA[3,4]             # 0 allocation (static array creation)

    sum(a) * sum(b)         # 0 allocation (scalars don't allocate)
end
```

```
julia> @btime foo($sx)
  1.821 ns (0 allocations: 0 bytes)
```

The decrease in memory allocations from SVectors is especially relevant for operations that result in temporary vectors, such as broadcasting.

VECTOR

```
x      = rand(10)


foo(x) = sum(2 .* x)
```

```
julia> @btime foo($x)
  36.695 ns (2 allocations: 144 bytes)
```

SVECTOR

```
x      = rand(10)
sx     = SVector(x...)

foo(x) = sum(2 .* x)
```

```
julia> @btime foo($sx)
  1.822 ns (0 allocations: 0 bytes)
```

Interestingly, the performance benefits of SVectors extend beyond memory allocation. This means that, even when operations on regular vectors don't allocate memory, SVectors could still deliver significant speedups. Below, we demonstrate this through the function `sum(f, <vector>)`, which adds all elements of `<vector>` after they're transformed via `f`. The example shows that the implementation with SVectors yields faster execution times, despite that regular vectors already don't incur memory allocations.

VECTOR

```
x      = rand(10)


foo(x) = sum(a -> 10 + 2a +  3a^2, x)
```

```
julia> @btime foo($x)
  9.420 ns (0 allocations: 0 bytes)
```

SVECTOR

```
x      = rand(10)
sx     = SVector(x...)

foo(x) = sum(a -> 10 + 2a +  3a^2, x)
```

```
julia> @btime foo($sx)
  1.822 ns (0 allocations: 0 bytes)
```

SVECTOR TYPE AND ITS MUTABLE VARIANT

SVectors are immutable, and so their elements can't be added, removed, or modified. To accommodate mutable collections, the package `StaticArrays` additionally provides a variant given by the `MVector` type. The creation of MVectors and their slices follow the same syntax as SVectors, but with the function `SVector` replaced with `MVector`. This is illustrated below.

IMMUTABLE STATIC VECTOR (SVECTOR)

```
x     = [1,2,3]
sx    = SVector(x...)

sx[1] = 0
```

```
ERROR: setindex!(::SVector{3, Int64}, value, ::Int) is not defined
```

MUTABLE STATIC VECTOR (MVECTOR)

```
x     = [1,2,3]
mx    = MVector(x...)

mx[1] = 0
```

```
julia> mx
3-element MVector{3, Int64} with indices SOneTo(3):
 0
 2
 3
```

The mutability of MVectors makes them ideal for initializing a vector that will eventually be filled via a for-loop. In fact, executing `similar(sx)` when `sx` is an SVector automatically returns an MVector.

'SIMILAR' FOR SVECTORS

```
sx = SVector(1,2,3)

mx = similar(sx)        # it defines an MVector with undef elements
```

```
julia> mx
3-element MVector{3, Int64} with indices SOneTo(3):
 129635981728016
 129635981728000
               2
```

TYPE STABILITY: SIZE IS PART OF THE STATIC VECTOR'S TYPE

SVectors are formally defined as objects with type `SVector{N,T}`, where `N` specifies the number of elements and `T` denotes the element's type. For instance, `SVector(4,5,6)` has type `SVector{3,Int64}`, indicating that it comprises 3 elements with type `Int64`. Importantly, this implies that **the number of elements is part of the `SVector` type**. This feature, which is shared with MVectors and inherited from tuples, can readily introduce type instabilities if not handled carefully.

To ensure type stability, you can employ approaches similar to those employed for tuples. Essentially, we should either pass SVectors and MVectors as function arguments, or dispatch by the number of elements through the `Val` technique.

TYPE UNSTABLE

```
x  = rand(50)


function foo(x)

    output = MVector{length(x), eltype(x)}(undef)

    for i in eachindex(x)
        temp      = x .> x[i]
        output[i] = sum(temp)
    end

    return output
end

@code_warntype foo(x)                    # type unstable
```

SVECTOR AS FUNCTION ARGUMENT (TYPE STABLE)

```
x  = rand(50)
sx = SVector(x...)

function foo(x)

    output = MVector{length(x), eltype(x)}(undef)

    for i in eachindex(x)
        temp      = x .> x[i]
        output[i] = sum(temp)
    end

    return output
end

@code_warntype foo(sx)                    # type stable
```

VAL (TYPE STABLE)

```
x  = rand(50)


function foo(x, ::Val{N}) where N
    sx     = SVector{N, eltype(x)}(x)
    output = MVector{N, eltype(x)}(undef)

    for i in eachindex(x)
        temp      = x .> x[i]
        output[i] = sum(temp)
    end

    return output
end

@code_warntype foo(x, Val(length(x)))     # type stable
```

PERFORMANCE COMPARISON

MVectors offer performance benefits over regular vectors. However, bear in mind that they're never more performant than SVectors. In fact, certain operations on MVectors may still trigger memory allocations. For this reason, the general guideline is to prefer SVectors when the collection doesn't need to be mutated, restricting MVectors when in-place mutation is necessary.

Below, we compare the performance of SVectors and MVectors. The examples demonstrate that they may exhibit similar performance, although SVectors are more performant in certain cases. Likewise, SVectors and MVectors consistently outperform regular vectors for small collections.

NO DIFFERENCE BETWEEN SVECTOR AND MVECTOR

```
x      = rand(10)
sx     = SVector(x...)
mx     = MVector(x...)

foo(x) = sum(a -> 10 + 2a +  3a^2, x)
```

```
julia> @btime foo($x)
  9.426 ns (0 allocations: 0 bytes)
julia> @btime foo($sx)
  1.821 ns (0 allocations: 0 bytes)
julia> @btime foo($mx)
  6.614 ns (0 allocations: 0 bytes)
```

DIFFERENCES

```
x      = rand(10)
sx     = SVector(x...)
mx     = MVector(x...)

foo(x) = 10 + 2x +  3x^2
```

```
julia> @btime foo.($x)
  35.112 ns (2 allocations: 144 bytes)
julia> @btime foo.($sx)
  1.909 ns (0 allocations: 0 bytes)
julia> @btime foo.($mx)
  14.586 ns (1 allocations: 96 bytes)
```

STATIC VECTORS VS PRE-ALLOCATIONS

Considering the advantages of static vectors over regular vectors, let's compare their performance to other strategies that decrease memory allocations. In particular, we'll examine how they stack up against **pre-allocating memory** for intermediate outputs. Our examples demonstrate that static vectors can efficiently store intermediate results, making pre-allocation techniques unnecessary. Moreover, the examples reveal that storing the final output in an MVector can lead to performance gains over using a regular vector.

For the illustration, consider a for-loop that requires an intermediate result during each iteration `i`. This involves counting the number of elements in `x` that are greater than `x[i]`, which can be formally implemented as `sum(x .> x[i])`. To make the comparison stark, we'll isolate the computation of the intermediate step `x .> x[i]`. Note that every implementation below requires pre-allocating the vector `output`, leaving us with only one decision to make: whether to pre-allocate the temporary vector `temp`. This also explains why all the implementations below involve at least one memory allocation.

NO PRE-ALLOCATION

```julia
x  = rand(50)


function foo(x; output = similar(x))
    for i in eachindex(x)
        temp      = x .> x[i]
        output[i] = sum(temp)
    end

    return output
end
```

```
julia> @btime foo($x)
  2.788 µs (152 allocations: 5.156 KiB)
```

PRE-ALLOCATION

```julia
x  = rand(50)


function foo(x; output = similar(x), temp = similar(x))
    for i in eachindex(x)
        @. temp      = x > x[i]
           output[i] = sum(temp)
    end

    return output
end
```

```
julia> @btime foo($x)
  1.562 µs (4 allocations: 960 bytes)
```

SVECTOR

```julia
x  = rand(50)
sx = SVector(x...)

function foo(x; output = Vector{Float64}(undef, length(x)))
    for i in eachindex(x)
        temp      = x .> x[i]
        output[i] = sum(temp)
    end

    return output
end
```

```
julia> @btime foo($sx)
  344.444 ns (2 allocations: 480 bytes)
```

SVECTOR AND MVECTOR ('SIMILAR')

```julia
x  = rand(50)
sx = SVector(x...)

function foo(x; output = similar(x))
    for i in eachindex(x)
        temp      = x .> x[i]
        output[i] = sum(temp)
    end

    return output
end
```

```
julia> @btime foo($sx)
  309.357 ns (1 allocations: 448 bytes)
```

SVECTOR AND MVECTOR

```julia
x  = rand(50)
sx = SVector(x...)

function foo(x; output = MVector{length(x),eltype(x)}(undef))
    for i in eachindex(x)
        temp      = x .> x[i]
        output[i] = sum(temp)
    end

    return output
end
```

```
julia> @btime foo($sx)
  308.000 ns (1 allocations: 448 bytes)
```

The "No-Preallocation" tab serves as a reference point for comparison with the other methods. As for the "Pre-allocating" tab, it reuses a regular vector to compute `temp`. In contrast, the "SVector" tab converts `x` to an SVector `sx` without pre-allocating `temp`. The benchmarks reveal that the latter approach is more performant, as it avoids memory allocation and benefits from additional optimizations provided by SVectors.

As for the last two tabs, they continue to define `x` as an SVector, but additionally treat `output` as an MVector. The last tab in particular does this by using `similar(sx)` to initialize `output`, whereas the other tab explicitly specifies an MVector. Comparing these cases, we observe that MVectors deliver additional performance gains.

FOOTNOTES

[1.] The recommended number of elements provided is actually lower than the documentation's suggested (100 elements). The reason for this discrepancy is that, as you approach the upper limit, the performance benefits of static vectors compared to regular vectors decrease significantly. As a result, the time spent benchmarking with collections of 100 elements will likely offset any potential advantage.

[2.] The approach based on `@SVector` requires some caveats. For instance, it doesn't support definition of SVectors based on local variables. In particular, it precludes the use of `eachindex(x)` within an array comprehension, unless `x` is a global variable.